WAPH-Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Team 10 Project

Project Topic/Title - Mini Facebook Web App

Team members

1. Member 1 - Corey Brunner, email: brunnecj@mail.uc.edu
2. Member 2 - Mike Wojtkiewicz, email: wojtkiml@mail.uc.edu
3. Member 3 - Bashar Rasras, email: rasrasbr@mail.uc.edu
4. Member 4 - Pawan Poudyel, email: poudyepn@mail.uc.edu

Project Management Information

Source code repository (private access): https://github.com/waph-uc-sm26-team10/waph-teamproject

Project homepage (public): homepage

Revision History

Date	Version	Description
07/09/2026	0.0	README Implementation
07/18/2026	0.0	Add SSL and Database setup
07/26/2026	0.1	Implement basic functional requirements such as login, registration, and account management.
07/27/2026	0.2	Add posts table and view-posts page, move all secrets to a git-ignored `.env`, deploy over HTTPS

Overview

The goal of this team project is to build a mini facebook application that takes into account security and full-stack web development.

A monolithic LAMP stack: Apache 2.4 over HTTPS, PHP 8.3, and MySQL. The code under `src/` is split into three layers - `Front-End/` (HTML pages and forms), `Back-End/` (request handling and session checks), and `Back-End/database.php` (all SQL, prepared statements only). `src/config/` holds configuration: `env.php` reads the git-ignored `.env`, `config.php` turns it into constants. No credentials appear in the source.

Note: You need to update this section according to each sprint

System Analysis

Create a full stack web application that creates, edits, and manages users. The application in sprint one should allow the user to create an account, manage that account, and login into the web app. Sprint 1 should account for viewing user posts.

High-level Requirements

Users can log in to a system, change passwords, edit their profile credentials, make and edit those posts, and view comments

There should be super users that can access the database directly, disable/enable and delete users.

Logged in users can have real time chat messages with other logged in users.

System Design

(Start from Sprint 1, keep updating)

Use-Case Realization

The use cases implemented in Sprint 1, and the files that realize each one:

Use case	Front-end	Back-end	Data layer function
Register a new account	`Front-End/registration form.php`	`Back-End/Registration.php`	`RegisterUser()`, `FindUser()`
Log in	`Front-End/loginform.php`	`Back-End/LoginUser.php`	`LoginUser()`
Change password	`Front-End/change password.php`	`Back-End/Change Password.php`	`Change Password()`

Use case	Front-end	Back-end	Data layer function
Edit profile (name, email, phone)	`Front-End/changeaccountinfo.php`	`Back-End/ChangeAccountInfo.php`	`UpdateUser()`
View posts	`Front-End/viewposts.php`	`Back-End/session_auth.php`	`GetAllPosts()`

`src/index.php` is the entry point. It reads the session and routes visitors to `Front-End/homepage.php` if they are logged in, or `Front-End/loginform.php` if not.

Database

Database schema https://github.com/waph-uc-sm26-team10/waph-teamproject/blob/main/database/schema.sql

Two tables in Sprint 1:

- `users` - `userid` (PK, auto-increment), `email` (unique), `username` (unique), `name`, `phone`, `password` (md5)
- `posts` - `postid` (PK, auto-increment), `title`, `content`, `date`, `owner` (FK → `users.userid`)

`userid` is a surrogate key. Usernames are editable on the account page, so using one as the primary key would mean every rename had to cascade into every table referencing that user. `userid` never changes; `username` and `email` are kept UNIQUE so neither can be duplicated. The foreign key is `ON DELETE CASCADE`, so deleting a user removes their posts.

User Interface

Click here to see front end/UI code # Implementation A monolithic application that intereacts with a database, the server, and the front-end side code. In sprint one it is broken down into 4 main tasks

- Interact with users table
  - Logging in
  - Registration
  - Account Management
- Create a posts table and allow users to view posts

Sprint 1 implementation notes

Configuration. Credentials live in a git-ignored `.env` read by `src/config/env.php`. The earlier `config.ini` and the committed `database-account.sql` password

are gone; the private key committed during Sprint 0 was purged from the git history and the certificate rotated.

Displaying posts. `GetAllPosts()` uses the prepared-statement pattern from Lecture 20 - prepare, execute, bind results, fetch:

```
$stmt = $mysqli->prepare("SELECT p.postid, p.title, p.content, p.date, u.username
                          FROM posts p JOIN users u ON p.owner = u.userid
                          ORDER BY p.date DESC, p.postid DESC");
$stmt->execute();
$stmt->bind_result($postid, $title, $content, $date, $owner);
while($stmt->fetch()){ $posts[] = ['title' => $title, /* ... */ ]; }
```

Values are encoded on output, so a post containing markup is displayed, not executed:

```
<h3><?php echo htmlentities($post['title'], ENT_QUOTES, 'UTF-8'); ?></h3>
```

Access control. `src/Back-End/session_auth.php` is included by every page requiring a login, and redirects to the login form when no session exists.

Deployment

Served by Apache over HTTPS at `https://waph-team10.minifacebook.com/`. Only the contents of `src/` go into the web root, so the schema, SSL folder and README are never web-reachable. `.env` is placed one directory *above* the web root - readable by PHP, not servable by Apache:

```
sudo rsync -a --delete ~/waph-teamproject/src/ /var/www/waph-teamproject/
sudo install -m 640 -o root -g www-data ~/waph-teamproject/.env /var/www/.env
```

Local Setup

Secrets are kept out of the repo via a git-ignored `.env` file. Copy the template and fill in real values:

```
cp .env.example .env   # then edit .env with your credentials
```

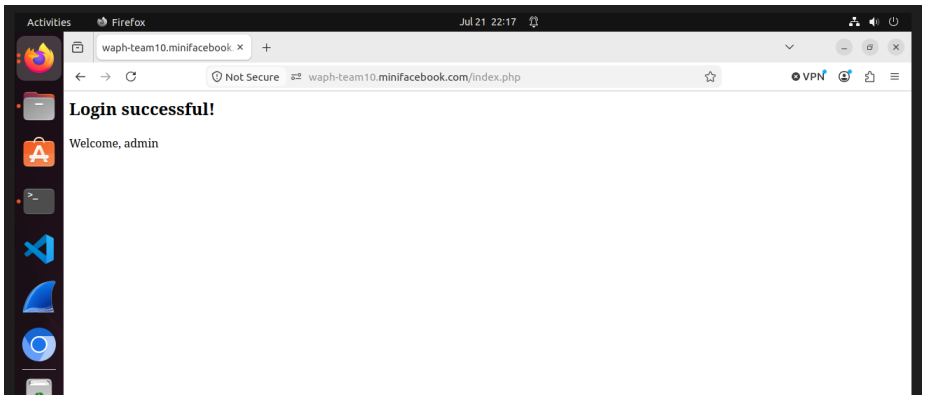
Database - run the setup script (reads `.env`, creates the DB/user, and seeds the admin account):

```
./database/setup.sh
```

SSL - generate a self-signed cert/key and install it (from the `sslkey/` folder):

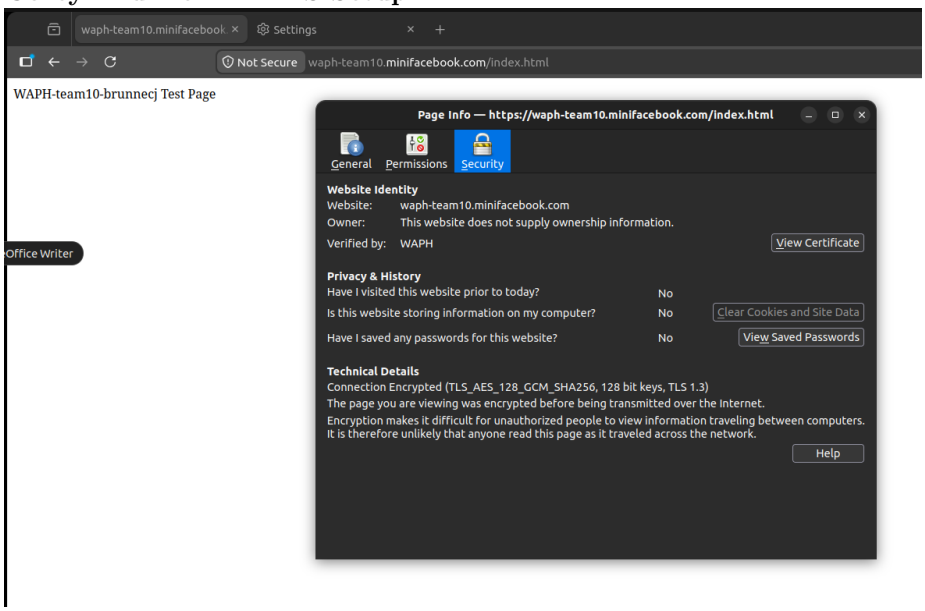
```
cd sslkey && ./ssl-keygen.sh
```

Login screenshot example

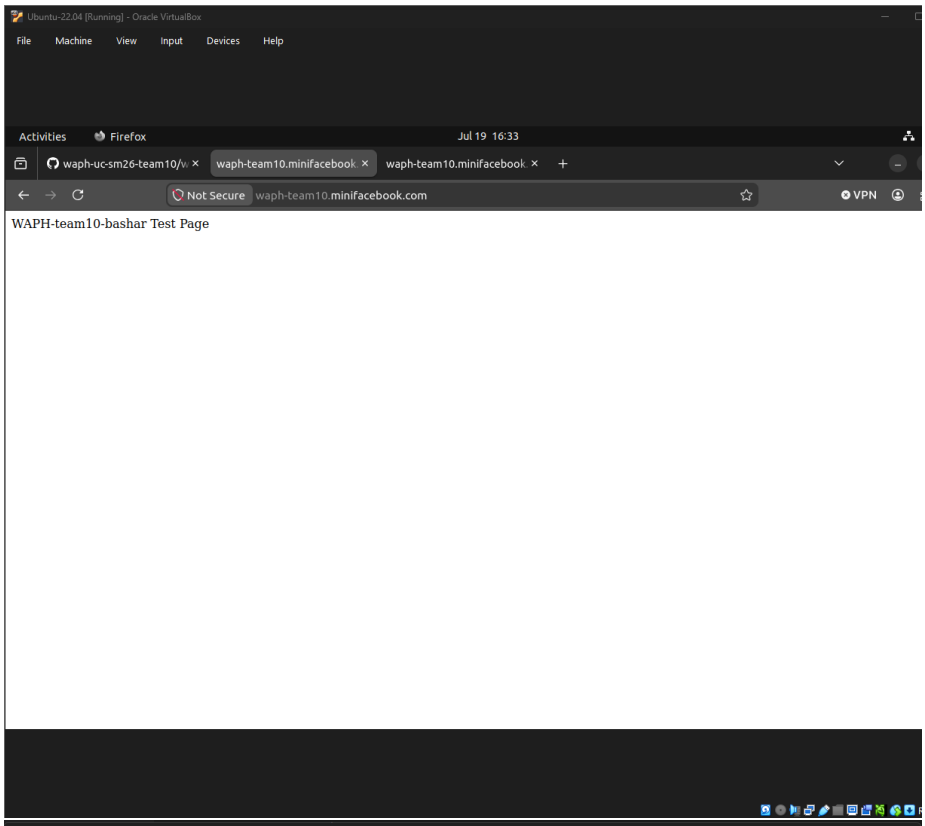


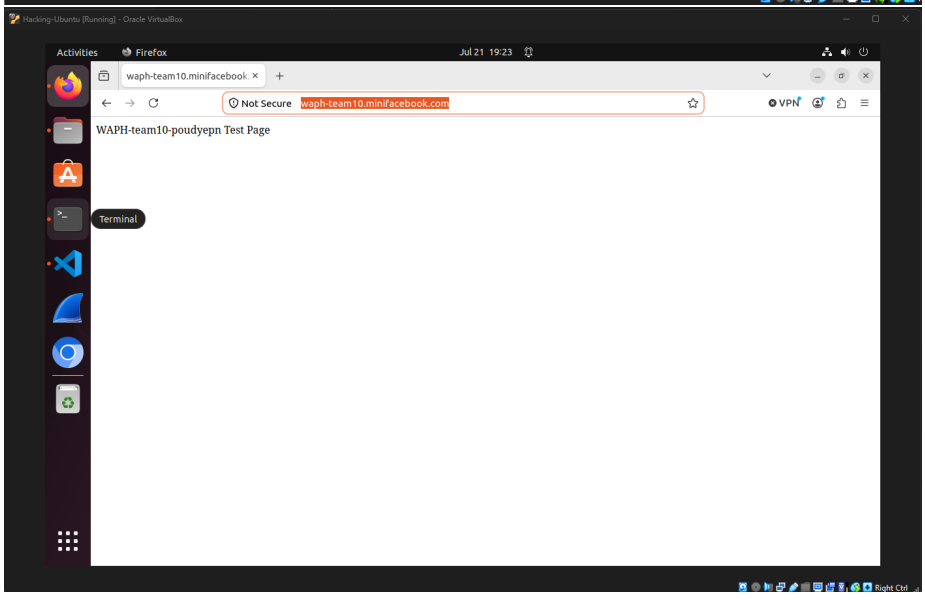
Local Domain Setup For Each Member

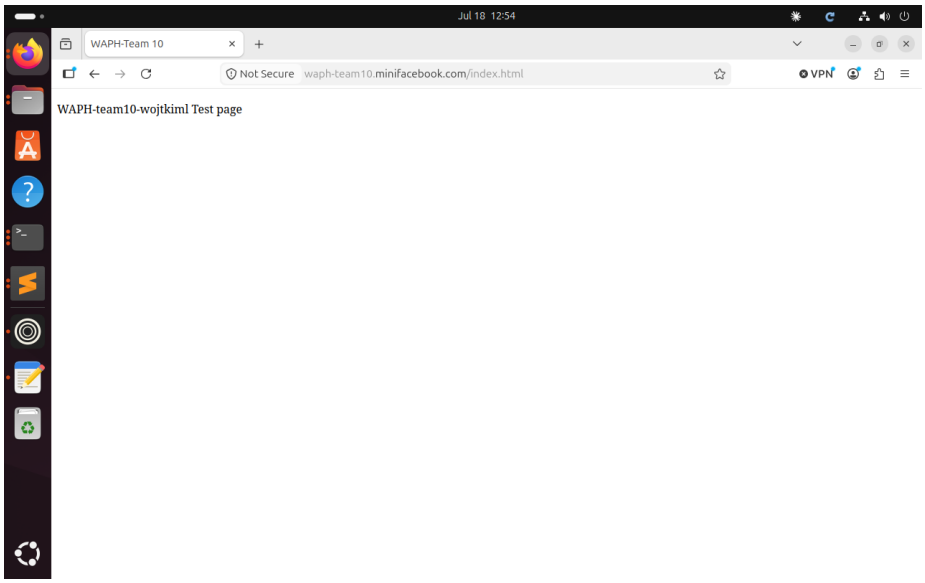
Corey Brunner HTTPS Setup



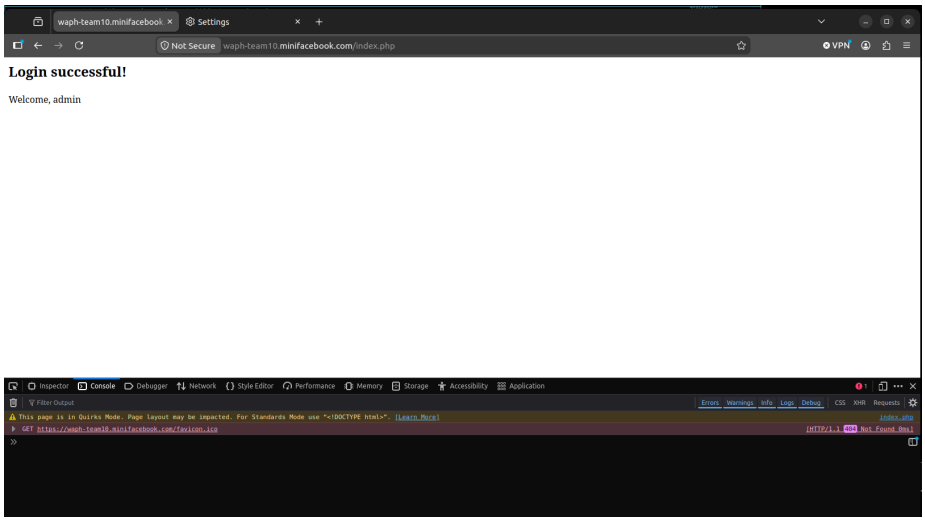
Bashar Rasras HTTPS Setup


Ubuntu-22.04 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Firefox Jul 19 16:33
waph-uc-sm26-team10/w
waph-team10.minifacebook
waph-team10.minifacebook
Not Secure waph-team10.minifacebook.com
VPN
WAPH-team10-bashar Test Page


Hacking-Ubuntu [Running] - Oracle VirtualBox
Activities Firefox Jul 21 19:23
waph-team10.minifacebook
Not Secure waph-team10.minifacebook.com
VPN
WAPH-team10-poudyepn Test Page
Terminal
Right Ctrl



Successful Local Domain Login Attempt



(Start from Sprint 1, keep updating. However, it is important to prepare the technology from Sprint 0)

For each new sprint cycle, update the implementation of your system (break it down into subsections). Please include some code snippets to illustrate the implementation.

Specify your team's development approach, including programming languages, databases, development, testing, and deployment environments.

Security analysis

Include a brief explanation of your implementation and the security aspects based on the following questions:

- How did you apply the security programming principles in your project?
- What database security principles have you used in your project?
- Is your code robust and defensive? How?
- How did you defend your code against known attacks such as XSS, SQL Injection, CSRF, Session Hijacking
- How do you separate the roles of super users and regular users?

Sprint 1

Security programming principles. Least privilege and no secrets in code. PHP connects with a dedicated MySQL account scoped to this database only with its password in a git-ignored `.env` outside the web root. A missing setting makes `env()` log the problem and return a generic `Configuration error.` rather than connecting with a blank password.

Database security. Every statement in `database.php` is a prepared statement with bound parameters; no input is concatenated into SQL. Passwords are stored as `md5()` hashes, never plaintext. A foreign key from `posts.owner` to `users.userid` enforces referential integrity, and `username`/`email` carry UNIQUE constraints so duplicates are rejected by the database, not just the application.

Robust and defensive? Partly. Database errors are caught, logged with `error_log()`, and shown to the user as a generic message; `display_errors` is off. Earlier versions echoed raw MySQL exceptions and dumped `$_POST` (including the plaintext password) to the page, but they have been removed. Input validation is the main gap.

Defenses against known attacks:

Attack	Status in Sprint 1
SQL Injection	**Defended** - prepared statements throughout.
XSS	**Defended on output** - all rendered values pass through `htmlentities(..., ENT_QUOTES, 'UTF-8')`. Verified with a post containing `<script>alert(1)</script>`.

Attack	Status in Sprint 1
Session Hijacking	**Partial** - HTTPS-only and protected pages require a session. Cookie flags and `session_regenerate_id()` not yet implemented.
CSRF	**Not implemented.** Planned for Sprint 2 using the Lecture 20 token pattern.

Super users vs regular users. Not yet implemented - Sprint 1 has one role. Sprint 2 plan: a `superusers` table with an `/admin` folder behind its own session check.

Gaps carried into Sprint 2: input validation (client and server), CSRF tokens, CSP headers, session cookie hardening, role-based access control, and an open-source CSS template.

Demo (screenshots)

You need to capture screenshots to demonstrate how your web application works. The screenshots must be accompanied by a short description of its functionalities following the implementation as below:

- Everyone can register a new account and then login
- Superuser can disable an account
  - The disabled account cannot log in
  - Superuser can enable the disabled account
  - The enabled user can log in
- A regular logged-in user can delete her own existing posts but cannot delete the posts of others
- CSRF attack to delete a post should be detected and prevented
- A regular logged-in user cannot access the link for superusers
- A logged-in user can have a real-time chat with other logged-in users

Sprint 1

Demonstrable this sprint: register, log in, change password, edit profile (name, email, phone), and view posts. The homepage and posts page require a session. Requesting them logged out redirects to the login form.

Super users, post creation/deletion, comments, CSRF demonstration, and real-time chat are not implemented yet and will be demonstrated in later sprints.

Software Process Management

(Start from Sprint 0, keep updating)

Introduce how your team uses a software management process, e.g., Scrum, and how your teamwork collaborates.

Scrum process

Sprint 0

Duration: 09/07/2026-21/07/2026

Completed Tasks:

1. Create Database Schema
2. Create SSL keys
3. Host Web server over https and custom domain

Contributions:

1. Corey Brunner, 7 commits, 4.5 hours, contributed in Included a custom domain screenshot inside readme. Completed Individual tasks.
2. Member 2, x commits, y hours, contributed in xxx
3. Member 3, x commits, y hours, contributed in xxx
4. Member 4, x commits, y hours, contributed in xxx

Sprint 1

Duration: 07/14/2026-07/28/2026

Completed Tasks: Sprint 1 required tasks:

1. Database design and implementation - `users` and `posts` tables
2. User registration and login
3. Logged-in users can change their password
4. Logged-in users can edit their profile - name, additional email, phone
5. Logged-in users can view posts from the database

Supporting tasks:

6. Update the README and project documentation
7. Capture and upload demo screenshots
8. Move all credentials into a git-ignored `.env`, purge the committed private key and database password from the git history, and rotate the TLS certificate
9. Deploy the application over HTTPS with only `src/` inside the web root

Contributions:

1. Corey Brunner, 6 commits, 9 hours, contributed in Created file structure to store files either in Front-End, Back-End, Database schema, and updated readme. Implemented basic code to allow users to sign on, create accounts, and manage accounts.
2. Member 2 - Mike Wojtkiewicz, 6 commits, 3 hours, contributed by moving all secrets to a git-ignored `.env` and purged the committed TLS key and database password from the git history; fixed the broken `ALTER TABLE` and rewrote `schema.sql` with the new `posts` table and a surrogate `userid` primary key; added `GetAllPosts()`, `viewposts.php`, `session_auth.php` and `index.php`; fixed a password change that reported success on a wrong old password, and a profile edit that rejected the user's own username; removed debug output that leaked plaintext passwords and SQL errors; deployed to Apache over HTTPS.
3. Member 3 - Bashar Rasras, x commits, y hours, contributed in xxx
4. Member 4 - Pawan Poudyel, 6 commits, y hours, contributed in task 1, 2

Sprint Retrospection: _(Introduction to Sprint Retrospection:

Working through the sprints is a continuous improvement process. Discussing the completed sprint can improve the next sprint walk through a much more efficient one. Sprint retrospection is done once a sprint is finished and the team is ready to start another sprint planning meeting. This discussion can take up to 1 hour depending on the ideal team size of 4 members. Discussing good things that happened during the sprint can improve the team's morale, good team collaboration, appreciating someone who did a fantastic job solving a blocker issue, work well-organized, and helping someone in need. This will improve the team's confidence and keep them motivated. As a team, we can discuss what has gone wrong during the sprint and come up with improvement points for the next sprints. Few points can be like, need to manage time well, need to prioritize the tasks properly and finish a task in time, incorrect design lead to multiple reviews and that wasted time during the sprint, team meetings were too long which consumed most of the effective work hours. We can mention every problem is in the sprint which is hindering the progress. Finally, this meeting should improve your next sprint drastically and understand the team dynamics well. Mention the bullet points and discuss how to solve it.)

Good	Could have been better	How to improve?
	communication and meeting times	Have a set time where everyone gets in a call to discuess who is working on what

Sprint x

Duration: MM/DD/YYYY-MM/DD/YYYY

Completed Tasks:

1. Task 1
2. Task 2
3. ...
4.

Contributions:

1. Member 1, x commits, y hours, contributed in xxx
2. Member 2, x commits, y hours, contributed in xxx
3. Member 3, x commits, y hours, contributed in xxx
4. Member 4, x commits, y hours, contributed in xxx

Sprint Retrospection:

Good	Could have been better	How to improve?

Appendix

Include the content (in text, not as images) of the SQL files and all source code of your PHP files (with the file name).